

Coletando dados na internet

Aula 16

Ana Júlia Lima



Desenvolvido para
Young 1 - sexta

Ctrl Play
Brasil

Sumário

1	Coletando Dados na Internet — Web Scraping	2
1.1	Introdução	2
1.2	Por que aprender Web Scraping?	2
1.3	Entendendo o HTML	2
1.4	Instalando bibliotecas	2
1.5	Acessando páginas web com Python	2
1.6	Usando o BeautifulSoup	3
1.7	Funções principais do BeautifulSoup	3
1.8	Exemplo completo de Web Scraping	3
1.9	Resumo prático	3
2	Pandas — Trabalhando com Dados em Python	4
2.1	Introdução	4
2.2	O que é o Pandas?	4
2.3	Instalação e importação	4
2.4	Criando um DataFrame	4
2.5	Relacionando Pandas e Web Scraping	4
2.6	Salvando dados em arquivo	5
2.7	Lendo arquivos com Pandas	5
2.8	Operações úteis com DataFrames	5
2.9	Resumo prático	5
3	Atividades — Web Scraping e Pandas	6
4	Desafio Final — Coletor de Dados Inteligente	7
5	Gabarito — Web Scraping e Pandas	8
6	Gabarito — Desafio Final	11
7	Mini Projeto Final — Sistema de Coleta e Análise de Citações	12
7.1	Objetivo	12
7.2	Código completo	12
7.3	Exemplo de uso	14
7.4	O que esse projeto ensina	15
7.5	Desafios extras	15

1 Coletando Dados na Internet — Web Scraping

1.1 Introdução

O **Web Scraping** é a técnica de coletar informações de sites de forma automática, usando código Python. Com ele, é possível capturar textos, links, tabelas e outros dados exibidos em uma página web.

1.2 Por que aprender Web Scraping?

- Automatiza a coleta de dados;
- Permite construir bases personalizadas;
- É útil para análises de preços, notícias, estatísticas;
- Ensina a compreender a estrutura das páginas HTML.

1.3 Entendendo o HTML

Antes de extrair dados, é importante compreender a estrutura do HTML, que é composta por **tags**, **atributos** e **conteúdo**.

```
<html>
  <head>
    <title>Exemplo de Página</title>
  </head>
  <body>
    <h1>Bem-vindo!</h1>
    <p class="descricao">Este é um exemplo simples de HTML.</p>
  </body>
</html>
```

1.4 Instalando bibliotecas

Para acessar páginas e analisar HTML, instale as bibliotecas:

```
pip install requests
pip install BeautifulSoup4
```

1.5 Acessando páginas web com Python

```
import requests

url = "https://www.example.com"
resposta = requests.get(url)

print(resposta.status_code) # código HTTP
print(resposta.text)        # HTML da página
```

Códigos HTTP comuns:

- 200 — Sucesso;
- 404 — Página não encontrada;
- 500 — Erro no servidor.

1.6 Usando o BeautifulSoup

```
from bs4 import BeautifulSoup

html = resposta.text
sopa = BeautifulSoup(html, "html.parser")
```

1.7 Funções principais do BeautifulSoup

`prettify()` Formata o HTML para visualização:

```
print(sopa.prettify())
```

`find()` Retorna a primeira ocorrência da tag especificada:

```
titulo = sopa.find("h1")
print(titulo.text)
```

`find_all()` Retorna uma lista com todas as ocorrências da tag:

```
paragrafos = sopa.find_all("p")
for p in paragrafos:
    print(p.text)
```

1.8 Exemplo completo de Web Scraping

```
import requests
from bs4 import BeautifulSoup

url = "https://www.example.com"
resposta = requests.get(url)

sopa = BeautifulSoup(resposta.text, "html.parser")

titulo = sopa.find("h1")
print("Título da página:", titulo.text)

paragrafos = sopa.find_all("p")
for p in paragrafos:
    print("Parágrafo:", p.text)
```

1.9 Resumo prático

- `requests`: acessa páginas web;
- `BeautifulSoup`: interpreta e navega no HTML;
- `find()`: retorna a primeira tag encontrada;
- `find_all()`: retorna todas as tags encontradas;
- `prettify()`: mostra o HTML formatado.

2 Pandas — Trabalhando com Dados em Python

2.1 Introdução

Depois de coletar informações com Web Scraping, é necessário organizar e analisar os dados. A biblioteca **Pandas** permite trabalhar com tabelas de dados (DataFrames) de forma simples e eficiente.

2.2 O que é o Pandas?

O Pandas (Python Data Analysis Library) é usado para:

- Organizar dados em formato de tabela;
- Importar e exportar arquivos (CSV, Excel, JSON etc.);
- Tratar valores nulos e duplicados;
- Filtrar, ordenar e agrupar informações;
- Fazer análises estatísticas com facilidade.

2.3 Instalação e importação

```
pip install pandas
```

```
import pandas as pd
```

2.4 Criando um DataFrame

```
import pandas as pd
```

```
dados = {  
    "Nome": ["Ana", "Bruno", "Carla"],  
    "Idade": [23, 30, 27],  
    "Cidade": ["São Paulo", "Rio de Janeiro", "Curitiba"]  
}
```

```
df = pd.DataFrame(dados)  
print(df)
```

Saída:

	Nome	Idade	Cidade
0	Ana	23	São Paulo
1	Bruno	30	Rio de Janeiro
2	Carla	27	Curitiba

2.5 Relacionando Pandas e Web Scraping

Após coletar dados com BeautifulSoup, podemos armazená-los em um DataFrame para análise.

```
import requests  
from bs4 import BeautifulSoup  
import pandas as pd  
  
url = "https://quotes.toscrape.com/"  
resposta = requests.get(url)  
sopa = BeautifulSoup(resposta.text, "html.parser")
```

```

citacoes = sopa.find_all("span", class_="text")
autores = sopa.find_all("small", class_="author")

lista_citacoes = [c.text for c in citacoes]
lista_autores = [a.text for a in autores]

df = pd.DataFrame({
    "Citação": lista_citacoes,
    "Autor": lista_autores
})

print(df.head())

```

2.6 Salvando dados em arquivo

```
df.to_csv("citacoes.csv", index=False, encoding="utf-8")
```

2.7 Lendo arquivos com Pandas

```

dados = pd.read_csv("citacoes.csv")
print(dados.head())

```

2.8 Operações úteis com DataFrames

- `df.head()` — mostra as 5 primeiras linhas;
- `df.info()` — mostra informações gerais;
- `df.describe()` — exibe estatísticas básicas;
- `df.shape` — número de linhas e colunas;
- `df.columns` — nomes das colunas;
- `df.sort_values()` — ordena os dados;
- `df.dropna()` — remove valores nulos;
- `df.fillna()` — substitui valores nulos.

2.9 Resumo prático

- **requests**: acessa páginas web;
- **BeautifulSoup**: interpreta HTML;
- **pandas**: organiza e analisa os dados coletados.

3 Atividades — Web Scraping e Pandas

Nível 1 — Conceitos básicos

1. Faça um programa que acesse o site: `https://www.example.com`
 - Mostre o código de status da página
 - Mostre o HTML completo
2. Usando o BeautifulSoup, capture o título da página (`<h1>`).
3. Capture todos os parágrafos (`<p>`) da página e imprima cada um.
4. Use o método `prettify()` para exibir o HTML formatado.

Nível 2 — Extraindo e organizando dados

1. Acesse o site: `https://quotes.toscrape.com/`
 - Capture todas as citações (`span class="text"`)
 - Capture todos os autores (`small class="author"`)
2. Armazene as citações e autores em duas listas separadas.
3. Crie um DataFrame com duas colunas:
 - Citação
 - Autor
4. Mostre apenas as 5 primeiras linhas usando `head()`.
5. Salve o DataFrame em um arquivo CSV chamado `citacoes.csv`.

Nível 3 — Manipulando dados com Pandas

1. Leia o arquivo `citacoes.csv` criado anteriormente.
2. Mostre:
 - Informações do DataFrame (`info()`)
 - Estatísticas (`describe()`)
3. Mostre o número de linhas e colunas usando `shape`.
4. Ordene os dados pelo nome do autor usando `sort_values()`.
5. Crie um novo DataFrame manual com:
 - Nome
 - Idade
 - Cidade
6. Filtre apenas as pessoas com idade maior que 25.

4 Desafio Final — Coletor de Dados Inteligente

Crie um programa completo que:

- Acesse um site de citações (`quotes.toscrape.com`)
- Extraia:
 - textos das citações
 - nomes dos autores
- Armazene os dados em um DataFrame
- Salve em um arquivo CSV
- Depois:
 - Leia o arquivo novamente
 - Mostre:
 - * total de citações
 - * lista de autores únicos

Objetivo: integrar Web Scraping + Pandas em um único fluxo de dados.

Desafios extras

- Contar quantas citações cada autor possui
- Ordenar autores por quantidade de citações
- Filtrar citações que contenham uma palavra específica
- Criar um gráfico simples (opcional)

5 Gabarito — Web Scraping e Pandas

Nível 1 — Conceitos básicos

1.

```
import requests

url = "https://www.example.com"
resposta = requests.get(url)

print(resposta.status_code)
print(resposta.text)

# Saída:
# 200
# <html>...</html>
```

2.

```
from bs4 import BeautifulSoup
import requests

url = "https://www.example.com"
resposta = requests.get(url)

sopa = BeautifulSoup(resposta.text, "html.parser")

titulo = sopa.find("h1")
print(titulo.text)

# Saída:
# Example Domain
```

3.

```
paragrafos = sopa.find_all("p")

for p in paragrafos:
    print(p.text)

# Saída:
# This domain is for use in illustrative examples...
# More information...
```

4.

```
print(sopa.prettify())

# Saída:
# HTML formatado (indentado)
```

Nível 2 — Extraíndo e organizando dados

1 e 2.

```

import requests
from bs4 import BeautifulSoup

url = "https://quotes.toscrape.com/"
resposta = requests.get(url)

sopa = BeautifulSoup(resposta.text, "html.parser")

citacoes = sopa.find_all("span", class_="text")
autores = sopa.find_all("small", class_="author")

lista_citacoes = [c.text for c in citacoes]
lista_autores = [a.text for a in autores]

```

3.

```

import pandas as pd

df = pd.DataFrame({
    "Citação": lista_citacoes,
    "Autor": lista_autores
})

print(df.head())

```

4.

```

print(df.head())

# Saída:
# primeiras 5 linhas da tabela

```

5.

```

df.to_csv("citacoes.csv", index=False, encoding="utf-8")

# Arquivo criado com sucesso

```

Nível 3 — Manipulando dados com Pandas

1.

```

import pandas as pd

df = pd.read_csv("citacoes.csv")
print(df.head())

```

2.

```

print(df.info())
print(df.describe())

# Saída:
# Informações das colunas e estatísticas

```

3.

```

print(df.shape)

# Saída:
# (linhas, colunas)

4.

df_ordenado = df.sort_values(by="Autor")

print(df_ordenado.head())

5.

dados = {
    "Nome": ["Ana", "Bruno", "Carla"],
    "Idade": [23, 30, 27],
    "Cidade": ["SP", "RJ", "PR"]
}

df2 = pd.DataFrame(dados)
print(df2)

6.

filtro = df2[df2["Idade"] > 25]

print(filtro)

# Saída:
# Bruno e Carla

```

6 Gabarito — Desafio Final

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://quotes.toscrape.com/"
resposta = requests.get(url)

sopa = BeautifulSoup(resposta.text, "html.parser")

citacoes = sopa.find_all("span", class_="text")
autores = sopa.find_all("small", class_="author")

lista_citacoes = [c.text for c in citacoes]
lista_autores = [a.text for a in autores]

df = pd.DataFrame({
    "Citação": lista_citacoes,
    "Autor": lista_autores
})

df.to_csv("citacoes.csv", index=False, encoding="utf-8")

# Lendo novamente
dados = pd.read_csv("citacoes.csv")

print("Total de citações:", len(dados))

autores_unicos = dados["Autor"].unique()
print("Autores únicos:", autores_unicos)

# Saída:
# Total de citações: 10 (ou mais)
# Autores únicos: ['Autor1', 'Autor2', ...]
```

Extras (opcional)

```
# Contar citações por autor
contagem = dados["Autor"].value_counts()
print(contagem)

# Filtrar citações com palavra específica
filtro = dados[dados["Citação"].str.contains("life")]
print(filtro)
```

7 Mini Projeto Final — Sistema de Coleta e Análise de Citações

Neste projeto, vamos criar um sistema completo que:

- Coleta citações da internet
- Armazena os dados
- Permite análise com menu interativo

7.1 Objetivo

O sistema deve permitir:

- Coletar dados do site
- Salvar em arquivo CSV
- Carregar os dados
- Exibir análises

7.2 Código completo

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

def coletar_dados():
    print("Coletando dados...")

    url = "https://quotes.toscrape.com/"
    resposta = requests.get(url)

    sopa = BeautifulSoup(resposta.text, "html.parser")

    citacoes = sopa.find_all("span", class_="text")
    autores = sopa.find_all("small", class_="author")

    lista_citacoes = [c.text for c in citacoes]
    lista_autores = [a.text for a in autores]

    df = pd.DataFrame({
        "Citação": lista_citacoes,
        "Autor": lista_autores
    })

    df.to_csv("citacoes.csv", index=False, encoding="utf-8")

    print("Dados coletados e salvos com sucesso!")

def carregar_dados():
    try:
        df = pd.read_csv("citacoes.csv")
        print("Dados carregados!")
        return df
```

```

except:
    print("Erro: arquivo não encontrado.")
    return None

def mostrar_dados(df):
    if df is not None:
        print(df.head())

def total_citacoes(df):
    if df is not None:
        print("Total de citações:", len(df))

def autores_unicos(df):
    if df is not None:
        print(df["Autor"].unique())

def top_autores(df):
    if df is not None:
        print(df["Autor"].value_counts())

def buscar_palavra(df):
    if df is not None:
        palavra = input("Digite uma palavra: ")
        resultado = df[df["Citação"].str.contains(palavra, case=False)]
        print(resultado)

def menu():
    df = None

    while True:

        print("\n--- MENU ---")
        print("1 - Coletar dados")
        print("2 - Carregar dados")
        print("3 - Mostrar dados")
        print("4 - Total de citações")
        print("5 - Autores únicos")
        print("6 - Top autores")
        print("7 - Buscar palavra")
        print("0 - Sair")

        opcao = input("Escolha: ")

        if opcao == "1":
            coletar_dados()

        elif opcao == "2":
            df = carregar_dados()

```

```

elif opcao == "3":
    mostrar_dados(df)

elif opcao == "4":
    total_citacoes(df)

elif opcao == "5":
    autores_unicos(df)

elif opcao == "6":
    top_autores(df)

elif opcao == "7":
    buscar_palavra(df)

elif opcao == "0":
    print("Saindo...")
    break

else:
    print("Opção inválida.")

if __name__ == "__main__":
    menu()

```

7.3 Exemplo de uso

```

--- MENU ---
1 - Coletar dados
2 - Carregar dados
3 - Mostrar dados
4 - Total de citações
5 - Autores únicos
6 - Top autores
7 - Buscar palavra
0 - Sair

Escolha: 1
Coletando dados...
Dados coletados e salvos com sucesso!

Escolha: 2
Dados carregados!

Escolha: 4
Total de citações: 10

Escolha: 6
Albert Einstein    2
Mark Twain        1
...

```

7.4 O que esse projeto ensina

- Web Scraping com `requests` e `BeautifulSoup`
- Manipulação de dados com `pandas`
- Leitura e escrita de arquivos CSV
- Estrutura de menu com `while`
- Organização de código em funções

7.5 Desafios extras

- Coletar múltiplas páginas automaticamente
- Salvar também as tags das citações
- Criar um ranking dos autores mais citados
- Exportar para Excel (`.xlsx`)
- Adicionar tratamento de erros com `try/except`